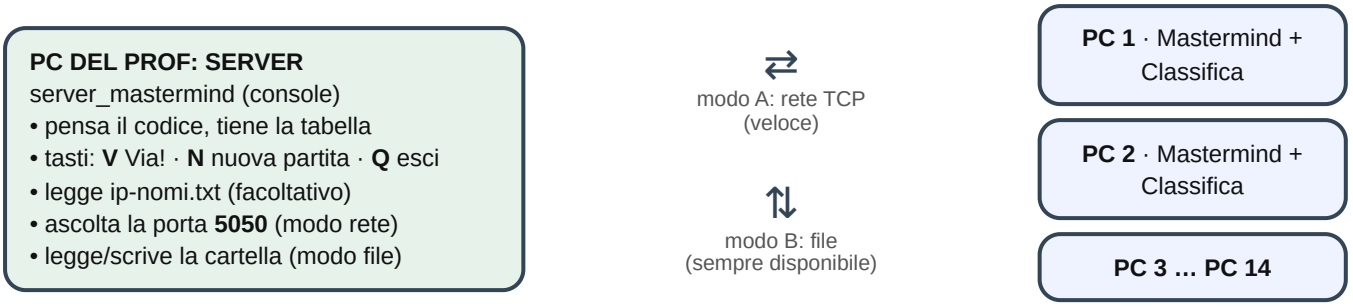
# Mastermind 2.0: tutta la classe contro un solo codice

**3INF · Lazarus / Free Pascal · versione 1.0 · 02/11/2026** · documento per il docente (non per i ragazzi: non è pubblicato né collegato alle pagine degli studenti)

> **In una frase.** Il PC del prof pensa UN codice di 4 cifre (1-6) e tiene la classifica; ogni ragazzo gioca il suo Mastermind normale ma contro quel codice, e nella scheda «Classifica» vede: *Migliore: N tentativi · Peggiore: N tentativi · Tu: N tentativi, posizione P di T*.

## 1. Lo schema




**CARTELLA CONDIVISA o repository GitHub** (modo B): `partita.txt` · `ranking.txt` · `giocatori/NOME.txt` (un file per studente: niente conflitti)

```
MASTERMIND 2.0 - SERVER   [V] Via!  [N] Nuova partita  [R] ridisegna  [I] ip-nomi  [Q] esci
Rete: in ascolto sulla porta 5050  Cartella: C:\mastermind\partita

PARTITA P3x64719 - IN GIOCO - finiti: 3 di 5 - migliore: 1 - peggiore: 3
 Pos  Nome                 Tentativi  Secondi  Stato
   1  Bruno C.                    1       20  INDOVINATO
   2  Anna D.                     2       10  INDOVINATO
   2  Dino F.                     2       10  INDOVINATO
   4  Carla G.                    3       30  INDOVINATO
   -  Fabio H.                    1        -  sta provando

CHI HA GIA' PROVATO / CHI NO (dalla lista ip-nomi.txt)
  192.168.10.21    Bruno C.            ha indovinato (1 tentativi)
  192.168.10.28    Ines L.             NON ha ancora provato
```

Così appare la console del server (nomi inventati).

## 2. Il protocollo in breve

Una riga per messaggio, campi separati da `;`. Rete: il client si collega, manda UNA riga, legge UNA riga, chiude. Dettaglio completo in `protocollo.md`.

| Direzione | Riga | Significato |
|---|---|---|
| client → server | `GUESS;NOME;1234` | un tentativo |
| client → server | `STATO;NOME` | solo aggiornare la console |
| server → client | `RESULT;esatte;spostate;tentativi;posizione;totale;migliore;peggiore` | tentativo sbagliato |

| | | |
|---|---|---|
| server → client | `WIN;NOME;tentativi;secondi;posizione;totale;migliore;peggiore` | indovinato (al posto di RESULT) |
| server → client | `WAIT` | il prof non ha ancora dato il Via |
| server → client | `ERR;motivo` | riga sbagliata |

**Posizione** = il tuo posto tra chi ha indovinato (0 = non ancora). Ordine: meno tentativi, poi meno secondi dal Via; stessi tentativi e secondi = pari merito (1, 2, 2, 4). **Migliore / peggiore** = tentativi del migliore e del peggiore tra chi ha finito. I secondi li misura il server.

| Esempio | Riga |
|---|---|
| tentativo sbagliato | `GUESS;Sara M.;1234` → `RESULT;1;2;1;0;0;0;0` |
| indovinato | `GUESS;Sara M.;4162` → `WIN;Sara M.;2;85;3;5;1;4` |
| prima del Via | `GUESS;Sara M.;1111` → `WAIT` |

### Modo B: i file

| File | Chi lo scrive | Formato |
|---|---|---|
| `partita.txt` | server | `PARTITA;id;seme;ATTESA` prima del Via (il seme non c'è); `PARTITA;id;seme;GIOCO;oraDelVia` dopo. Dal seme ogni client ricava lo stesso codice. |
| `giocatori/NOME.txt` | ogni studente | una riga per tentativo: `id;NOME;1234;secondi` |
| `ranking.txt` | server (ogni 2 s se cambia) | `PARTITA`, `STATO`, `TOTALE`, `MIGLIORE`, `PEGGIORE`, poi righe `R;posizione;nome;tentativi;secondi;S\|N` |
| `comando.txt` | il prof (facoltativo) | `VIA`, `NUOVA` oppure `FINE`; il server lo legge e lo cancella |

> **Limite del modo file con GitHub.** Studente: Commit + Push · prof: Pull · il server aggiorna · prof: Commit + Push di ranking.txt · studente: Pull. Ritardo di **almeno un minuto**: è il piano di riserva, NON si gioca dal vivo così. Con una cartella di rete il ritardo è di pochi secondi. Inoltre nel modo file il codice si ricava dal seme (un ragazzo bravo potrebbe leggerlo): il server ricalcola comunque ogni tentativo.

## 3. Come si fa in classe, passo per passo

### 3.1 Prima della lezione, sul PC del prof (10 minuti, meglio il giorno prima)

| N. | Dove | Cosa fare |
|---|---|---|
| 1 | Esplora file | Crea la cartella `C:\mastermind` (o in Documenti) e copiaci i 4 file `mm_logica.pas`, `mm_file.pas`, `mm_tcp.pas`, `server_mastermind.pas`. |
| 2 | Lazarus, menu | **File** → **Apri…** → scegli `server_mastermind.pas` → il programma si apre come progetto (se chiede, conferma «Crea progetto da file»; tipo: **Applicazione console / Programma semplice**). |
| 3 | Lazarus, menu | **Esegui** → **Compila** (oppure F9). Se compare l'errore «Can't find unit ssockets», vedi la tabella «cosa può andare storto» (riga 1). |
| 4 | Esplora file | Nella cartella ora c'è `server_mastermind.exe`: è il tuo server. Doppio clic per avviarlo. |
| 5 | Finestra nera del server | Alla prima partenza Windows chiede «Windows Defender Firewall ha bloccato alcune funzionalità». Metti la spunta su **Reti private** e premi **Consenti accesso** (vedi 3.3). Se chiede una password di amministratore, NON insistere: si usa il modo file (3.4). |
| 6 | Windows, bottone Start | Trova il tuo IP senza comandi: **Start** → **Impostazioni** (l'ingranaggio) → **Rete e Internet** → **Ethernet** (o **Wi-Fi**) → clic sul nome della connessione → in basso **Proprietà** → riga **Indirizzo IPv4**. Con IP statici (la lista nome/IP che hai) è sempre lo stesso: scrivilo. |
| 7 | Blocco note | Facoltativo: crea `ip-nomi.txt` accanto al server con una riga per PC (esempio al punto 3.5). Nella finestra del server premi **I** per rileggerlo. |

### 3.2 Sui PC dei ragazzi (una volta sola, ~10 minuti: si fa guidati dal docente)

> Si parte dal Mastermind normale già fatto (progetto `mastermind` in Documenti). Si lavora su una COPIA della cartella: «Mastermind 2». Il percorso è pensato per ragazzi che hanno già finito il gioco; niente riga di comando.

| N. | Dove | Cosa fare (e in bengalese per il PC 14) |
|---|---|---|
| 1 | Esplora file | Copia la cartella del progetto `mastermind` e chiamala `mastermind2`. Dentro metti i 4 file del Mastermind 2.0 (`mm_logica.pas`, `mm_file.pas`, `mm_tcp.pas`, `client_unit.pas`) che il prof ti dà.<br>> mastermind ফোল্ডারের একটা কপি করে নাম দাও mastermind2। ভেতরে ৪টা ফাইল রাখো। |
| 2 | Lazarus, menu | **Progetto** → **Apri progetto…** → `mastermind2` → il file `.lpi`. |
| 3 | Lazarus, scheda **Gioco** | Palette **Standard** → clicca **TEdit** → clicca accanto al tuo tentativo: nell'Ispettore oggetti, **Name** = `Edit2`, **Text** = il tuo nome (per esempio «Sara M.»). Questo è il tuo nome in classifica: scegli tu come comparire.<br>> Standard থেকে TEdit নিয়ে Gioco ট্যাবে বসাও। Name = Edit2, Text = তোমার নাম। |
| 4 | Lazarus, scheda **Record** | Cambia il titolo della scheda in **Classifica** (proprietà **Caption** di TabSheet3). La `Label4` è già lì: mostrerà la console. |
| 5 | Lazarus, editor del codice | Apri `unit1.pas`, **Ctrl + A**, **Ctrl + V** incollando il contenuto di `unit1_client.pas` (te lo dà il prof). **F2** per salvare.<br>> unit1.pas খোলো, সব মুছে unit1_client.pas-এর কোড বসাও। F2 দিয়ে সেভ করো। |
| 6 | Blocco note | Nella cartella `mastermind2` crea un file di testo `server.txt` con una sola riga: l'IP del PC del prof e la porta, per esempio `192.168.10.5:5050`. Se il prof ha scelto i file, crea `cartella.txt` con il percorso della cartella condivisa. |

| | | |
|---|---|---|
| 7 | Lazarus | **F9**. FATTO! Scrivi 4 cifre, premi **Prova**: prima del Via leggi «Aspetta il Via del prof…». Quando il prof preme V, si gioca.<br>> F9 চাপো। প্রফেসর V চাপলে খেলা শুরু। |

### 3.3 Il firewall: tre vie d'uscita

> Windows può bloccare le connessioni IN ARRIVO sul PC del prof. Chi si collega è il ragazzo (connessione in uscita): di norma non è bloccata. Il problema è solo sul PC del prof, la prima volta che il server si mette in ascolto.

| Via | Cosa fare | Serve l'amministratore? |
|---|---|---|
| 1. «Consenti accesso» | Alla prima partenza compare la finestra del firewall: spunta **Reti private** → **Consenti accesso**. Vale per sempre per quel programma. | A volte sì (chiede la password). Se non la puoi dare, passa alla via 3. |
| 2. Un'altra porta o un altro PC | Prova `server_mastermind --porta=5051` (e `:5051` in server.txt). Oppure avvia il server su un altro PC dove l'accesso è già consentito o dove c'è un amministratore. Una volta sola il sysadmin può aprire la porta 5050 TCP in ingresso. | Dipende dal PC |
| 3. Modo file | Non serve nessuna rete: si usa la cartella condivisa o GitHub (3.4). È già il piano di riserva automatico dei client. | No |

**Per sapere subito se la rete funziona:** sul PC di un ragazzo scrivi l'IP in `server.txt` e premi Prova. Se compare «Problema: server non raggiungibile…» il firewall (o l'IP) è il colpevole: il client passa da solo ai file se c'è `cartella.txt`.

### 3.4 Piano file, passo per passo

| N. | Chi | Cosa fare |
|---|---|---|
| 1 | Prof | Avvia il server con la cartella scelta: crea un collegamento al server e nelle Proprietà aggiungi dopo il nome `--cartella=Z:\mastermind` (cartella di rete) oppure il percorso del repository clonato. Con `--senza-rete` la rete resta spenta e il firewall non chiede niente. |
| 2 | Prof | Preme **V**: il server scrive `partita.txt` (con il seme) e `ranking.txt`. |
| 3 | Ragazzi | `cartella.txt` accanto al programma, con la stessa cartella. Cartella di rete: i file arrivano da soli. Repository GitHub: **GitHub Desktop** → **Fetch origin** / **Pull** per ricevere, **Commit to main** + **Push origin** per mandare il proprio file in `giocatori`. |
| 4 | Prof | Con GitHub: **Pull** in GitHub Desktop → il server rilegge entro 2 secondi → **Commit** + **Push** di `ranking.txt`. |

> **Durante la lezione** (modo rete): nella finestra del server premi **V** per il Via; guarda la classifica e la lista «chi non ha ancora provato»; **N** per ricominciare con un codice nuovo (tutti tornano in attesa); **Q** per uscire. Senza tastiera: scrivi `VIA`, `NUOVA` o `FINE` in `comando.txt` nella cartella del server.

### 3.5 Esempio di ip-nomi.txt (dati inventati)

Una riga per PC: `IP;NOME`. Le righe che cominciano con # sono commenti. Con questo file il nome in classifica è quello scritto qui, anche se il ragazzo scrive un altro nome.

```
# ip-nomi.txt - esempio con dati finti
192.168.10.21;Bruno C.
192.168.10.22;Anna D.
192.168.10.23;Carla G.
192.168.10.24;Dino F.
192.168.10.28;Ines L.
```

Il server prende il nome dall'IP di chi si collega; per un IP che non c'è nella lista vale il nome scritto dal client. Attenzione: serve che la lista sia degli IP dei PC, non delle persone (se un ragazzo cambia PC cambia nome in classifica).

## 4. Cosa può andare storto: piano B e piano C

| N. | Problema | Piano B | Piano C |
|---|---|---|---|
| 1 | Lazarus non trova `ssockets` («Can't find unit ssockets») | Progetto → Opzioni progetto → Percorsi: aggiungi in «Altri file unit» la cartella `fpc\3.x.x\units\x86_64-win64\fcl-net` dentro la cartella di Lazarus (cerca `ssockets.ppu`) | Modo file: non usa `ssockets` se non si include `mm_tcp` |
| 2 | Il firewall blocca l'ingresso sul PC del prof | «Consenti accesso» su Reti private, oppure altra porta | Modo file (cartella di rete o GitHub) |
| 3 | I ragazzi vedono «server non raggiungibile» | Controlla IP e porta in `server.txt` (Impostazioni → Rete); il server è acceso e dice «in ascolto»? | Il client passa da solo ai file se c'è `cartella.txt` |
| 4 | Il server dice «Rete: NON attiva» | Porta già usata: `--porta=5051` (e cambia server.txt) | `--senza-rete` + modo file |
| 5 | Cartella di rete non raggiungibile | Cartella locale sul PC del prof condivisa con i ragazzi | Repository GitHub con GitHub Desktop (ritardo ≥ 1 minuto) |
| 6 | Due ragazzi con lo stesso nome | Il server li conta come uno: usare ip-nomi.txt o nomi diversi («Sara M.», «Sara T.») | — |
| 7 | Orologi dei PC sballati (solo modo file) | Conta il numero di tentativi; i secondi sono indicativi | — |
| 8 | Un ragazzo legge il codice (modo file ricava il codice dal seme) | Il server ricalcola e conta i tentativi veri: è un gioco di fiducia | Per la verifica vera usa il modo rete: il codice non esce mai dal PC del prof |
| 9 | Il server si chiude per sbaglio | Riavvialo: si ricomincia con codice nuovo (la classifica si azzera) | Gli ultimi risultati restano in `ranking.txt` |
| 10 | La tastiera del server non risponde | Clicca sulla finestra nera; oppure `comando.txt` con VIA / NUOVA | — |

## 5. Che cosa è stato provato davvero e che cosa no

**Provato con fpc 3.2.2 (Linux), tutto passato: 104 prove automatiche.** 1) la logica pura con orologio finto (conteggio esatte/spostate, WAIT prima del Via, tentativo giusto e sbagliato, classifica con pari merito, migliore e peggiore, chi arriva tardi, nuova partita, IP→nome); 2) il trasporto a file con server e 5 client sulla stessa cartella (righe di partite vecchie, riga a metà, nessun conteggio doppio, file di configurazione); 3) la rete TCP su localhost con il VERO `server_mastermind` e client simulati (anche con richieste in un thread, riga senza senso, client muto, passaggio automatico ai file, nome preso dall'IP); 4) la scheda dello studente (`unit1_client.pas`) compila contro finte unità LCL.

**NON provato:** 1) le vere finestre LCL di Lazarus (la scheda dello studente è stata compilata solo contro finte unità: sintassi e tipi, non il comportamento a schermo); 2) il server e i client su Windows (provato solo su Linux; il codice usa solo `ssockets` e `sockets` che esistono anche su Windows); 3) il firewall e la rete reali della scuola; 4) la tastiera `Crt` sulla console di Windows (provata solo su una console Linux); 5) il lavoro con GitHub Desktop; 6) 14 PC insieme (provate poche connessioni simultanee). **Non è stato usato lNet, né Indy, né Synapse:** solo `ssockets`, che è già dentro Free Pascal/Lazarus (nessun pacchetto da installare, nessuna ricostruzione dell'IDE). La rete gira in un thread (`TThread`), così la finestra non si blocca.

**Consiglio:** prima di usarlo con i ragazzi, 10 minuti di prova con due PC della scuola (server + un client).

### Risultati delle prove (uscita dei programmi di prova, nomi inventati)

```
MASTERMIND 2.0 - risultati delle prove (02/11/2026 07:09) - fpc 3.2.2

##### 1. LOGICA PURA (test_logica)
== Valutazione esatte/spostate (stessa logica del Mastermind normale)
  OK   1234 vs 1234 = 4 esatte
  OK   1234 vs 4321 = 0 esatte 4 spostate
  OK   1123 vs 1111 = 2 esatte 0 spostate
  OK   1234 vs 1355 = 1 esatta 1 spostata
  OK   1122 vs 2211 = 0 esatte 4 spostate
  OK   codice valido 1234
  OK   codice non valido 1237 / 123 / abcd / 12345
  OK   stesso seme = stesso codice
  OK   codice da seme: 4 cifre da 1 a 6
== Partita (orologio finto)
  OK   prima del Via: WAIT  [WAIT]
  OK   prima del Via nessun giocatore registrato
  OK   codice non valido  [ERR;codice non valido (servono 4 cifre da 1 a 6)]
  OK   nome mancante  [ERR;nome non valido]
  OK   comando sconosciuto  [ERR;comando sconosciuto]
  OK   tentativo sbagliato: RESULT
  OK     ... 1 tentativo, posizione 0, totale 0
  OK   Anna indovina al 2o tentativo dopo 10 s  [WIN;Anna;2;10;1;1;2;2]
  OK   Bruno al 1o tentativo dopo 20 s: primo  [WIN;Bruno;1;20;1;2;1;2]
  OK   Anna ora e' seconda
  OK   Dino pari merito con Anna (2 tentativi, 10 s): posizione 2
  OK     ... e Anna resta seconda
  OK   Ennio (2 tentativi, 15 s) e' quarto (dopo il pari merito)
  OK   Carla: 3 tentativi, posizione 5 di 5
  OK   migliore = 1 tentativo, peggiore = 3 tentativi
  OK   Fabio (non finito): 1 tentativo, posizione 0, totale resta 5
  OK   Anna riprova dopo aver vinto: resta WIN con 2 tentativi  [WIN;Anna;2;10;2;5;1;3]
  OK   nomi senza distinzione maiuscole: "anna" e' Anna
== Chi arriva tardi
  OK   Gina entra molto dopo e indovina subito: WIN dopo 1000 s, seconda  [WIN;Gina;1;1000;2;6;1;3]
  OK     ... Anna e Dino scalano al 3o posto, Ennio 5o, Carla 6a
  OK   STATO di chi non ha ancora provato (Hugo)  [STATO;GIOCO;0;0;6;1;3]
== Console dello studente
  OK   testo console  [Migliore: 1 tentativi · Peggiore: 3 tentativi · Tu: 3 tentativi, posizione 6 di 6]
  OK   testo console chi non ha finito  [Migliore: 1 tentativi · Peggiore: 3 tentativi · Tu: 1 tentativi, non hai anco
ra indovinato (hanno finito in 6)]
  OK   ParsaRisposta capisce WIN
  OK   ParsaRisposta capisce WAIT
  OK   ParsaRisposta rifiuta rumore
== File ranking.txt riletto da un client
  OK   ranking.txt contiene la riga di Carla
  OK   ranking.txt: MIGLIORE;1 e PEGGIORE;3
  OK   partita.txt dopo il Via contiene il seme
== Nuova partita
  OK   tabella vuota, di nuovo in attesa, codice nuovo
  OK   WAIT di nuovo  [WAIT]
  OK   partita.txt in attesa NON rivela il seme
== IP -> nome (ip-nomi.txt)
  OK   IP noto: il nome scritto dal client viene ignorato
  OK   IP sconosciuto: vale il nome scritto
  OK   pannello presenze: Sara non ha ancora provato
  OK     ... e Luca sta provando

RISULTATO: 45 prove OK su 45, fallite: 0

##### 2. TRASPORTO FILE (test_file): server + 5 client sulla stessa cartella
== Prima del Via
  OK   partita.txt in attesa NON contiene il seme
  OK   il client risponde WAIT prima del Via
  OK     ... e non scrive nulla
  OK   console in attesa: nessuno ha indovinato
== Via!
  OK   partita.txt ora contiene il seme
  OK   ERR se il codice e' scritto male
  OK   Anna sbaglia: RESULT, 1 tentativo
  OK   Anna indovina: WIN, 2 tentativi, 10 secondi
  OK   Bruno indovina subito: WIN, 1 tentativo, 20 s
  OK   Carla: WIN con 3 tentativi
  OK   Anna dopo la vittoria: il suo file resta di 2 righe
== Il server rilegge la cartella
  OK   Poll vede 8 tentativi nuovi (2+1+3+2)
  OK   un secondo Poll non conta due volte
  OK   tabella del server: Anna 2 tentativi 10 s
  OK   Dino pari merito con Anna (posizione 2)
  OK   Carla e' quarta di 4
  OK   migliore 1, peggiore 3
== I client leggono ranking.txt
  OK   console di Carla  [Migliore: 1 tentativi · Peggiore: 3 tentativi · Tu: 3 tentativi, posizione 4 di 4]
  OK   console di Anna  [Migliore: 1 tentativi · Peggiore: 3 tentativi · Tu: 2 tentativi, posizione 2 di 4]
== Chi arriva tardi
```

```
  OK   Gina entra tardi e indovina: WIN, 1 tentativo, 2000 s (conta dal Via)
  OK   console di Gina  [Migliore: 1 tentativi · Peggiore: 3 tentativi · Tu: 1 tentativi, posizione 2 di 5]
== Righe strane
  OK   riga di una partita vecchia (altro id): ignorata
  OK   riga a meta' (senza a capo): non ancora letta
  OK   quando arriva l'a capo la riga conta; il server ha ricalcolato: e' una vittoria
== Nuova partita
  OK   dopo Nuova partita le vecchie righe non contano
  OK   client: WAIT di nuovo
== File di configurazione accanto al programma
  OK   solo cartella.txt: modo file
  OK   server.txt IP:PORTA: modo rete
  OK   server.txt solo IP: porta predefinita 5050
  OK   nessun file: non configurato

RISULTATO: 30 prove OK su 30, fallite: 0

##### 3. RETE TCP su localhost (server_mastermind vero + client simulati)
######## PROVE A: server senza ip-nomi.txt (porta 5077)
== Prima del Via (rete)
  OK   STATO risponde ATTESA
  OK   GUESS prima del Via: WAIT
  OK   riga cruda  [WAIT]
  OK   riga senza senso: ERR, il server non muore
  OK   server ancora vivo dopo la riga strana
== Via!
  OK   dopo il comando Via lo STATO e' GIOCO
  OK   Anna sbaglia: RESULT, 1 tentativo, posizione 0
  OK   Anna indovina: WIN al 2o tentativo, posizione 1 di 1
  OK     ... ha impiegato almeno 1 secondo
  OK   Bruno indovina al 1o: primo, Anna scende a seconda
  OK   Bruno vede migliore 1 e peggiore 2
  OK   richiesta ASINCRONA (thread) completata con CheckSynchronize
  OK   Carla (async): WIN a 3 tentativi, ultima
  OK   console di Carla  [Migliore: 1 tentativi · Peggiore: 3 tentativi · Tu: 3 tentativi, posizione 3 di 3]
  OK   Dino 2 tentativi come Anna ma piu' lento: dietro Anna
  OK   Gina (ritardataria) indovina al 1o: seconda per tentativi
  OK   dopo aver vinto, ripetere non cambia i tentativi
  OK   server ancora vivo dopo un client muto
== Server spento: il client passa da solo ai file (piano B)
  OK   rete non raggiungibile -> modo file automatico
  OK   il server ha letto Ivo dal file e lo mette in ranking.txt
== ranking.txt scritto dal server per chi usa i file
  OK   contiene Bruno primo
  OK   contiene Carla
  OK   MIGLIORE;1 PEGGIORE;3
== Nuova partita (comando)
  OK   dopo NUOVA: di nuovo ATTESA
  OK     ... e Anna non ha piu' tentativi

RISULTATO: 25 prove OK su 25, fallite: 0
######## PROVE B: server con ip-nomi.txt (porta 5078)
== IP noto -> nome dalla tabella ip-nomi.txt
  OK   GUESS accettato
  OK   WIN
  OK   ranking.txt ha il nome VERO "Mario Rossi"
  OK     ... e NON il nome falso scritto dal client

RISULTATO: 4 prove OK su 4, fallite: 0


------------------------------------------------

MASTERMIND 2.0 - SERVER   [V] Via!  [N] Nuova partita  [R] ridisegna  [I] ip-nom

i  [Q] esci

Rete: in ascolto sulla porta 5077  Cartella: /tmp/claude-0/-home-user/b09df496-8

25b-5f09-8a08-1a0a65c53666/scratchpad/o4/cartA


PARTITA P3x65003 - IN ATTESA DEL VIA - finiti: 0 di 0 - migliore: 0 - peggiore:

0

 Pos  Nome                  Tentativi  Secondi  Stato

 (nessuno ha ancora provato)


CHI HA GIA' PROVATO / CHI NO (dalla lista ip-nomi.txt)

  (nessun file ip-nomi.txt: si usano i nomi scritti dai ragazzi)


##### 4. CONTROLLO DI COMPILAZIONE della scheda dello studente (unit1_client.pas) contro finte unita' LCL
  OK   unit1_client.pas compila (sintassi e tipi; NON e' una prova con la vera LCL)
```

```
TUTTO OK
```

## 6. Il codice completo

I file sono nella cartella `classe-3/mastermind-2/` del repository. Si compila con Lazarus/FPC senza pacchetti aggiuntivi.

### mm_logica.pas

Il cervello (nessuna finestra, nessuna rete): codice, conteggio, tabella, classifica, protocollo

```
unit mm_logica;
{ MASTERMIND 2.0 - LOGICA (nessuna finestra, nessuna rete).
  Qui c'e' il cervello: il codice segreto, il conteggio esatte/spostate,
  la tabella dei giocatori, la classifica e il "linguaggio" a righe di testo.
  Si puo' provare da solo, senza Lazarus e senza rete. }

{$mode objfpc}{$H+}

interface

uses
  SysUtils, Classes;

type
  TFase = (faAttesa, faGioco);
  TEsito = (esAttesa, esNonValido, esOk, esVinto, esGiaVinto);

  { Quello che serve alla "console" dello studente }
  TSintesi = record
    Tentativi: Integer;   // tentativi fatti da te
    Posizione: Integer;   // il tuo posto tra chi ha finito (0 = non hai ancora finito)
    Totale: Integer;      // quanti hanno finito
    Migliore: Integer;    // tentativi del migliore (0 = nessuno ha ancora finito)
    Peggiore: Integer;    // tentativi del peggiore tra chi ha finito
  end;

  { Una riga di risposta del server, gia' letta }
  TRisposta = record
    Tipo: String;         // RESULT, WIN, WAIT, STATO, ERR
    Esatte, Spostate: Integer;
    Secondi: Integer;     // solo per WIN
    Sintesi: TSintesi;
    Messaggio: String;    // testo dell'errore (ERR)
  end;

  TGiocatore = class
    Nome: String;
    Tentativi: Integer;
    Risolto: Boolean;
    Secondi: Integer;     // secondi dal "Via!" alla vittoria
  end;

  TFunzioneSecondi = function: Double;   // orologio (in secondi), sostituibile nei test

  TMMTabella = class
  private
    FGiocatori: TList;
    FSegreto: String;
    FSeme: Int64;
    FFase: TFase;
    FIdPartita: String;
    FViaOra: Double;
    FCS: TRTLCriticalSection;
    FNumPartite: Integer;
    function Cerca(const ANome: String): TGiocatore;
    function Crea(const ANome: String): TGiocatore;
    function PosizioneDi(G: TGiocatore): Integer;
    function GetCount: Integer;
    function GetGiocatore(I: Integer): TGiocatore;
  public
    Orologio: TFunzioneSecondi;
    IpNomi: TStringList;       // righe IP=NOME (da ip-nomi.txt), facoltativo
    procedure CaricaIpNomi(const Testo: String);   // righe  IP;NOME
    function NomePerIp(const Ip: String): String;
    function TestoPresenze: String;   // chi ha provato / chi no (in base a ip-nomi)
    procedure Blocca;      // il thread di rete e quello principale non si pestano i piedi
    procedure Sblocca;
    constructor Create;
    destructor Destroy; override;
    procedure NuovaPartita(ASeme: Int64 = -1);
    procedure Via;
    function Tentativo(const ANome, AGuess: String; ASecondiForzati: Integer;
      out Esatte, Spostate: Integer): TEsito;
```

```
    function SintesiDi(const ANome: String): TSintesi;
    function Gestisci(const Linea: String; const IpClient: String = ''): String;
    function TestoClassifica: String;      // per la console del docente
    function TestoRanking: String;         // contenuto di ranking.txt
    function TestoPartita: String;         // contenuto di partita.txt
    property Fase: TFase read FFase;
    property IdPartita: String read FIdPartita;
    property Segreto: String read FSegreto;
    property Seme: Int64 read FSeme;
    property Count: Integer read GetCount;
    property Giocatori[I: Integer]: TGiocatore read GetGiocatore;
  end;

{ Funzioni "pure" (usate anche dai client) }
function CodiceDaSeme(ASeme: Int64): String;
procedure Valuta(const Segreto, Tent: String; out Esatte, Spostate: Integer);
function CodiceValido(const S: String): Boolean;
function PulisciNome(const S: String): String;
function ParsaRisposta(const Linea: String; out R: TRisposta): Boolean;
function SintesiDaRanking(const Testo, ANome: String; out Fase: TFase;
  out Id: String; out Seme: Int64): TSintesi;
function TestoConsole(const S: TSintesi): String;
function SecondiOra: Double;
function Campo(const Linea: String; N: Integer): String;   // N parte da 1

implementation

function SecondiOra: Double;
begin
  Result := Now * 86400.0;
end;

{ Il campo numero N (da 1) di una riga separata da punti e virgola }
function Campo(const Linea: String; N: Integer): String;
var
  i, k, ini: Integer;
begin
  Result := '';
  k := 1;
  ini := 1;
  for i := 1 to Length(Linea) + 1 do
    if (i > Length(Linea)) or (Linea[i] = ';') then
    begin
      if k = N then
      begin
        Result := Copy(Linea, ini, i - ini);
        Exit;
      end;
      k := k + 1;
      ini := i + 1;
    end;
end;

{ Da un numero (seme) esce sempre lo stesso codice: cosi' server e client
  del piano B (file) possono avere lo stesso codice senza parlarsi. }
function CodiceDaSeme(ASeme: Int64): String;
var
  x: Int64;
  i: Integer;
begin
  x := ASeme and $7FFFFFFF;
  Result := '';
  for i := 1 to 4 do
  begin
    x := (x * 1103515245 + 12345) and $7FFFFFFF;
    Result := Result + IntToStr(((x shr 16) mod 6) + 1);
  end;
end;

{ Stessa logica del Mastermind normale: prima le esatte, poi le spostate }
procedure Valuta(const Segreto, Tent: String; out Esatte, Spostate: Integer);
var
  i, j: Integer;
  usataS, usataT: array[1..4] of Boolean;
begin
  for i := 1 to 4 do
  begin
    usataS[i] := False;
    usataT[i] := False;
  end;
  Esatte := 0;
  for i := 1 to 4 do
    if Tent[i] = Segreto[i] then
    begin
      Esatte := Esatte + 1;
      usataS[i] := True;
      usataT[i] := True;
    end;
  Spostate := 0;
  for i := 1 to 4 do
```

```
    if not usataT[i] then
      for j := 1 to 4 do
        if (not usataS[j]) and (Tent[i] = Segreto[j]) then
        begin
          Spostate := Spostate + 1;
          usataS[j] := True;
          Break;
        end;
end;

function CodiceValido(const S: String): Boolean;
var
  i: Integer;
begin
  Result := Length(S) = 4;
  if Result then
    for i := 1 to 4 do
      if not (S[i] in ['1'..'6']) then
        Result := False;
end;

{ Toglie ; e caratteri strani dal nome (il ; separa i campi!), max 20 }
function PulisciNome(const S: String): String;
var
  i: Integer;
begin
  Result := '';
  for i := 1 to Length(S) do
    if (S[i] >= ' ') and not (S[i] in [';', '/', '\', ':', '*', '?', '"', '<', '>', '|']) then
      Result := Result + S[i];
  Result := Trim(Result);
  if Length(Result) > 20 then
    Result := Trim(Copy(Result, 1, 20));
end;

function SIntero(const S: String; Pred: Integer = 0): Integer;
begin
  Result := StrToIntDef(Trim(S), Pred);
end;

{ ------------------------------------------------------------ tabella }

constructor TMMTabella.Create;
begin
  inherited Create;
  InitCriticalSection(FCS);
  IpNomi := TStringList.Create;
  FGiocatori := TList.Create;
  Orologio := @SecondiOra;
  NuovaPartita;
end;

destructor TMMTabella.Destroy;
var
  i: Integer;
begin
  for i := 0 to FGiocatori.Count - 1 do
    TGiocatore(FGiocatori[i]).Free;
  FGiocatori.Free;
  IpNomi.Free;
  DoneCriticalSection(FCS);
  inherited Destroy;
end;

procedure TMMTabella.Blocca;
begin
  EnterCriticalSection(FCS);
end;

procedure TMMTabella.Sblocca;
begin
  LeaveCriticalSection(FCS);
end;

function TMMTabella.GetCount: Integer;
begin
  Result := FGiocatori.Count;
end;

function TMMTabella.GetGiocatore(I: Integer): TGiocatore;
begin
  Result := TGiocatore(FGiocatori[I]);
end;

function TMMTabella.Cerca(const ANome: String): TGiocatore;
var
  i: Integer;
begin
  Result := nil;
  for i := 0 to FGiocatori.Count - 1 do
```

```
    if SameText(TGiocatore(FGiocatori[i]).Nome, ANome) then
    begin
      Result := TGiocatore(FGiocatori[i]);
      Exit;
    end;
end;

function TMMTabella.Crea(const ANome: String): TGiocatore;
begin
  Result := TGiocatore.Create;
  Result.Nome := ANome;
  FGiocatori.Add(Result);
end;

procedure TMMTabella.NuovaPartita(ASeme: Int64);
var
  i: Integer;
begin
  for i := 0 to FGiocatori.Count - 1 do
    TGiocatore(FGiocatori[i]).Free;
  FGiocatori.Clear;
  if ASeme < 0 then
  begin
    Randomize;
    ASeme := Random(2000000000) + 1;
  end;
  FSeme := ASeme;
  FSegreto := CodiceDaSeme(FSeme);
  FFase := faAttesa;
  FNumPartite := FNumPartite + 1;      // l'id NON deve dipendere dal seme (rivelerebbe il codice)
  FIdPartita := 'P' + IntToStr(FNumPartite) + 'x' + IntToStr(Trunc(Orologio()) mod 100000);
  FViaOra := 0;
end;

procedure TMMTabella.Via;
begin
  if FFase = faAttesa then
  begin
    FFase := faGioco;
    FViaOra := Orologio();
  end;
end;

{ Chi e' meglio di G? Meglio = piu' risolti, a parita' meno tentativi, poi meno secondi.
  La posizione e' 1 + il numero di chi ha finito ed e' STRETTAMENTE meglio.
  Pari tentativi e pari secondi = stessa posizione (parimerito). }
function TMMTabella.PosizioneDi(G: TGiocatore): Integer;
var
  i: Integer;
  H: TGiocatore;
begin
  Result := 0;
  if not G.Risolto then Exit;
  Result := 1;
  for i := 0 to FGiocatori.Count - 1 do
  begin
    H := TGiocatore(FGiocatori[i]);
    if H.Risolto and ((H.Tentativi < G.Tentativi) or
       ((H.Tentativi = G.Tentativi) and (H.Secondi < G.Secondi))) then
      Result := Result + 1;
  end;
end;

function TMMTabella.Tentativo(const ANome, AGuess: String; ASecondiForzati: Integer;
  out Esatte, Spostate: Integer): TEsito;
var
  G: TGiocatore;
  nome: String;
begin
  Esatte := 0;
  Spostate := 0;
  nome := PulisciNome(ANome);
  if (nome = '') or not CodiceValido(AGuess) then
  begin
    Result := esNonValido;
    Exit;
  end;
  if FFase = faAttesa then
  begin
    Result := esAttesa;
    Exit;
  end;
  G := Cerca(nome);
  if G = nil then G := Crea(nome);
  Valuta(FSegreto, AGuess, Esatte, Spostate);
  if G.Risolto then
  begin
    Result := esGiaVinto;     // non conta piu': il risultato resta quello di prima
    Exit;
  end;
```

```
  G.Tentativi := G.Tentativi + 1;
  if Esatte = 4 then
  begin
    G.Risolto := True;
    if ASecondiForzati >= 0 then
      G.Secondi := ASecondiForzati
    else
      G.Secondi := Round(Orologio() - FViaOra);
    Result := esVinto;
  end
  else
    Result := esOk;
end;

function TMMTabella.SintesiDi(const ANome: String): TSintesi;
var
  i: Integer;
  G, H: TGiocatore;
begin
  Result.Tentativi := 0;
  Result.Posizione := 0;
  Result.Totale := 0;
  Result.Migliore := 0;
  Result.Peggiore := 0;
  for i := 0 to FGiocatori.Count - 1 do
  begin
    H := TGiocatore(FGiocatori[i]);
    if H.Risolto then
    begin
      Result.Totale := Result.Totale + 1;
      if (Result.Migliore = 0) or (H.Tentativi < Result.Migliore) then
        Result.Migliore := H.Tentativi;
      if H.Tentativi > Result.Peggiore then
        Result.Peggiore := H.Tentativi;
    end;
  end;
  G := Cerca(PulisciNome(ANome));
  if G <> nil then
  begin
    Result.Tentativi := G.Tentativi;
    Result.Posizione := PosizioneDi(G);
  end;
end;

function SintesiTesto(const S: TSintesi): String;
begin
  Result := IntToStr(S.Tentativi) + ';' + IntToStr(S.Posizione) + ';' +
    IntToStr(S.Totale) + ';' + IntToStr(S.Migliore) + ';' + IntToStr(S.Peggiore);
end;

{ IL PROTOCOLLO A RIGHE (vedi protocollo.md). Riceve UNA riga, restituisce UNA riga. }
function TMMTabella.Gestisci(const Linea: String; const IpClient: String): String;
var
  cmd, nome, guess: String;
  esa, spo: Integer;
  S: TSintesi;
  G: TGiocatore;
begin
  cmd := UpperCase(Trim(Campo(Linea, 1)));
  nome := PulisciNome(Campo(Linea, 2));
  if (IpClient <> '') and (NomePerIp(IpClient) <> '') then
    nome := NomePerIp(IpClient);        // IP noto: vale il nome della tabella, non quello scritto dal client
  if cmd = 'GUESS' then
  begin
    guess := Trim(Campo(Linea, 3));
    case Tentativo(nome, guess, -1, esa, spo) of
      esAttesa: Result := 'WAIT';
      esNonValido:
        if nome = '' then Result := 'ERR;nome non valido'
        else Result := 'ERR;codice non valido (servono 4 cifre da 1 a 6)';
      esOk:
        begin
          S := SintesiDi(nome);
          Result := 'RESULT;' + IntToStr(esa) + ';' + IntToStr(spo) + ';' + SintesiTesto(S);
        end;
      esVinto, esGiaVinto:
        begin
          S := SintesiDi(nome);
          G := Cerca(nome);
          Result := 'WIN;' + nome + ';' + IntToStr(G.Tentativi) + ';' + IntToStr(G.Secondi) +
            ';' + IntToStr(S.Posizione) + ';' + IntToStr(S.Totale) + ';' +
            IntToStr(S.Migliore) + ';' + IntToStr(S.Peggiore);
        end;
    end;
  end
  else if cmd = 'STATO' then
  begin
    S := SintesiDi(nome);
    if FFase = faAttesa then Result := 'STATO;ATTESA;' else Result := 'STATO;GIOCO;';
    Result := Result + SintesiTesto(S);
```

```
  end
  else
    Result := 'ERR;comando sconosciuto';
end;

procedure TMMTabella.CaricaIpNomi(const Testo: String);
var
  L: TStringList;
  i: Integer;
  ip, nm: String;
begin
  IpNomi.Clear;
  L := TStringList.Create;
  try
    L.Text := Testo;
    for i := 0 to L.Count - 1 do
    begin
      ip := Trim(Campo(L[i], 1));
      nm := PulisciNome(Campo(L[i], 2));
      if (ip <> '') and (ip[1] <> '#') and (nm <> '') then
        IpNomi.Add(ip + '=' + nm);
    end;
  finally
    L.Free;
  end;
end;

function TMMTabella.NomePerIp(const Ip: String): String;
begin
  Result := IpNomi.Values[Trim(Ip)];
end;

function TMMTabella.TestoPresenze: String;
var
  i: Integer;
  nm, st: String;
  G: TGiocatore;
  nessuno: Boolean;
begin
  Result := 'CHI HA GIA'' PROVATO / CHI NO (dalla lista ip-nomi.txt)' + LineEnding;
  nessuno := True;
  for i := 0 to IpNomi.Count - 1 do
  begin
    nm := IpNomi.ValueFromIndex[i];
    G := Cerca(nm);
    if G = nil then st := 'NON ha ancora provato'
    else if G.Risolto then st := 'ha indovinato (' + IntToStr(G.Tentativi) + ' tentativi)'
    else st := 'sta provando (' + IntToStr(G.Tentativi) + ' tentativi)';
    Result := Result + Format('  %-16s %-20s %s', [IpNomi.Names[i], nm, st]) + LineEnding;
    nessuno := False;
  end;
  if nessuno then Result := Result + '  (nessun file ip-nomi.txt: si usano i nomi scritti dai ragazzi)' + LineEnding;
end;

function TMMTabella.TestoClassifica: String;
var
  i, k, pos, n: Integer;
  lista: TList;
  G, H: TGiocatore;
  S: TSintesi;
  rig: String;
begin
  lista := TList.Create;
  try
    for i := 0 to FGiocatori.Count - 1 do lista.Add(FGiocatori[i]);
    // ordinamento a bolle: risolti prima, poi tentativi, poi secondi
    for i := 0 to lista.Count - 2 do
      for k := 0 to lista.Count - 2 - i do
      begin
        G := TGiocatore(lista[k]);
        H := TGiocatore(lista[k + 1]);
        if (H.Risolto and not G.Risolto) or
           ((H.Risolto = G.Risolto) and ((H.Tentativi < G.Tentativi) or
           ((H.Tentativi = G.Tentativi) and H.Risolto and (H.Secondi < G.Secondi)))) then
          lista.Exchange(k, k + 1);
      end;
    S := SintesiDi('');
    if FFase = faAttesa then rig := 'IN ATTESA DEL VIA' else rig := 'IN GIOCO';
    Result := 'PARTITA ' + FIdPartita + ' - ' + rig + ' - finiti: ' + IntToStr(S.Totale) +
      ' di ' + IntToStr(FGiocatori.Count) + ' - migliore: ' + IntToStr(S.Migliore) +
      ' - peggiore: ' + IntToStr(S.Peggiore) + LineEnding;
    Result := Result + ' Pos  Nome                 Tentativi  Secondi  Stato' + LineEnding;
    n := 0;
    for i := 0 to lista.Count - 1 do
    begin
      G := TGiocatore(lista[i]);
      pos := PosizioneDi(G);
      if pos > 0 then rig := IntToStr(pos) else rig := '-';
      Result := Result + Format(' %3s  %-20s  %9d  %7s  %s', [rig, G.Nome, G.Tentativi,
        BoolToStr(G.Risolto, IntToStr(G.Secondi), '-'),
```

```
        BoolToStr(G.Risolto, 'INDOVINATO', 'sta provando')]) + LineEnding;
      n := n + 1;
    end;
    if n = 0 then Result := Result + ' (nessuno ha ancora provato)' + LineEnding;
  finally
    lista.Free;
  end;
end;

{ ranking.txt: una riga per giocatore, ordinata (vedi protocollo.md, parte B) }
function TMMTabella.TestoRanking: String;
var
  i, k: Integer;
  lista: TList;
  G, H: TGiocatore;
  S: TSintesi;
begin
  lista := TList.Create;
  try
    for i := 0 to FGiocatori.Count - 1 do lista.Add(FGiocatori[i]);
    for i := 0 to lista.Count - 2 do
      for k := 0 to lista.Count - 2 - i do
      begin
        G := TGiocatore(lista[k]);
        H := TGiocatore(lista[k + 1]);
        if (H.Risolto and not G.Risolto) or
           ((H.Risolto = G.Risolto) and ((H.Tentativi < G.Tentativi) or
           ((H.Tentativi = G.Tentativi) and H.Risolto and (H.Secondi < G.Secondi)))) then
          lista.Exchange(k, k + 1);
      end;
    S := SintesiDi('');
    Result := 'PARTITA;' + FIdPartita + LineEnding;
    if FFase = faAttesa then Result := Result + 'STATO;ATTESA' + LineEnding
    else Result := Result + 'STATO;GIOCO' + LineEnding;
    Result := Result + 'TOTALE;' + IntToStr(S.Totale) + LineEnding +
      'MIGLIORE;' + IntToStr(S.Migliore) + LineEnding +
      'PEGGIORE;' + IntToStr(S.Peggiore) + LineEnding;
    for i := 0 to lista.Count - 1 do
    begin
      G := TGiocatore(lista[i]);
      Result := Result + 'R;' + IntToStr(PosizioneDi(G)) + ';' + G.Nome + ';' +
        IntToStr(G.Tentativi) + ';' + IntToStr(G.Secondi) + ';' +
        BoolToStr(G.Risolto, 'S', 'N') + LineEnding;
    end;
  finally
    lista.Free;
  end;
end;

{ partita.txt: il seme (e quindi il codice) si scrive SOLO dopo il Via }
function TMMTabella.TestoPartita: String;
begin
  if FFase = faAttesa then
    Result := 'PARTITA;' + FIdPartita + ';0;ATTESA' + LineEnding
  else
    Result := 'PARTITA;' + FIdPartita + ';' + IntToStr(FSeme) + ';GIOCO;' + IntToStr(Round(FViaOra)) + LineEnding;
end;

{ ------------------------------------------------------------ lato client }

function ParsaRisposta(const Linea: String; out R: TRisposta): Boolean;
var
  cmd: String;
begin
  FillChar(R, SizeOf(R), 0);
  R.Tipo := '';
  R.Messaggio := '';
  cmd := UpperCase(Trim(Campo(Linea, 1)));
  Result := True;
  if cmd = 'RESULT' then
  begin
    R.Tipo := 'RESULT';
    R.Esatte := SIntero(Campo(Linea, 2));
    R.Spostate := SIntero(Campo(Linea, 3));
    R.Sintesi.Tentativi := SIntero(Campo(Linea, 4));
    R.Sintesi.Posizione := SIntero(Campo(Linea, 5));
    R.Sintesi.Totale := SIntero(Campo(Linea, 6));
    R.Sintesi.Migliore := SIntero(Campo(Linea, 7));
    R.Sintesi.Peggiore := SIntero(Campo(Linea, 8));
  end
  else if cmd = 'WIN' then
  begin
    R.Tipo := 'WIN';
    R.Esatte := 4;
    R.Sintesi.Tentativi := SIntero(Campo(Linea, 3));
    R.Secondi := SIntero(Campo(Linea, 4));
    R.Sintesi.Posizione := SIntero(Campo(Linea, 5));
    R.Sintesi.Totale := SIntero(Campo(Linea, 6));
    R.Sintesi.Migliore := SIntero(Campo(Linea, 7));
    R.Sintesi.Peggiore := SIntero(Campo(Linea, 8));
```

```
  end
  else if cmd = 'WAIT' then
    R.Tipo := 'WAIT'
  else if cmd = 'STATO' then
  begin
    R.Tipo := 'STATO';
    R.Messaggio := Campo(Linea, 2);          // ATTESA o GIOCO
    R.Sintesi.Tentativi := SIntero(Campo(Linea, 3));
    R.Sintesi.Posizione := SIntero(Campo(Linea, 4));
    R.Sintesi.Totale := SIntero(Campo(Linea, 5));
    R.Sintesi.Migliore := SIntero(Campo(Linea, 6));
    R.Sintesi.Peggiore := SIntero(Campo(Linea, 7));
  end
  else if cmd = 'ERR' then
  begin
    R.Tipo := 'ERR';
    R.Messaggio := Campo(Linea, 2);
  end
  else
  begin
    R.Tipo := 'ERR';
    R.Messaggio := 'risposta non capita';
    Result := False;
  end;
end;

{ Legge ranking.txt (piano B): tira fuori la sintesi per il nome dato }
function SintesiDaRanking(const Testo, ANome: String; out Fase: TFase;
  out Id: String; out Seme: Int64): TSintesi;
var
  L: TStringList;
  i, pos, num: Integer;
  riga, nome: String;
  mio: Boolean;
begin
  FillChar(Result, SizeOf(Result), 0);
  Fase := faAttesa;
  Id := '';
  Seme := 0;
  nome := PulisciNome(ANome);
  L := TStringList.Create;
  try
    L.Text := Testo;
    for i := 0 to L.Count - 1 do
    begin
      riga := Trim(L[i]);
      if Copy(riga, 1, 8) = 'PARTITA;' then
      begin
        Id := Campo(riga, 2);
        if Campo(riga, 3) <> '' then Seme := StrToInt64Def(Campo(riga, 3), 0);
        if Campo(riga, 4) = 'GIOCO' then Fase := faGioco;      // forma di partita.txt
      end
      else if riga = 'STATO;GIOCO' then Fase := faGioco
      else if Copy(riga, 1, 7) = 'TOTALE;' then Result.Totale := SIntero(Campo(riga, 2))
      else if Copy(riga, 1, 9) = 'MIGLIORE;' then Result.Migliore := SIntero(Campo(riga, 2))
      else if Copy(riga, 1, 9) = 'PEGGIORE;' then Result.Peggiore := SIntero(Campo(riga, 2))
      else if Copy(riga, 1, 2) = 'R;' then
      begin
        pos := SIntero(Campo(riga, 2));
        mio := SameText(Campo(riga, 3), nome);
        num := SIntero(Campo(riga, 4));
        if mio then
        begin
          Result.Tentativi := num;
          Result.Posizione := pos;
        end;
      end;
    end;
  finally
    L.Free;
  end;
end;

function TestoConsole(const S: TSintesi): String;
begin
  Result := '';
  if S.Totale = 0 then
    Result := 'Nessuno ha ancora indovinato'
  else
    Result := 'Migliore: ' + IntToStr(S.Migliore) + ' tentativi ' + #194#183 +
      ' Peggiore: ' + IntToStr(S.Peggiore) + ' tentativi';
  Result := Result + ' ' + #194#183 + ' Tu: ' + IntToStr(S.Tentativi) + ' tentativi';
  if S.Posizione > 0 then
    Result := Result + ', posizione ' + IntToStr(S.Posizione) + ' di ' + IntToStr(S.Totale)
  else
    Result := Result + ', non hai ancora indovinato (hanno finito in ' + IntToStr(S.Totale) + ')';
end;

end.
```

## mm_file.pas

Trasporto B: file

```
unit mm_file;
{ MASTERMIND 2.0 - TRASPORTO B: FILE - e' il modo PREDEFINITO (non chiede permessi di rete).
  Cartella condivisa (cartella di rete, oppure repository GitHub clonato e sincronizzato):
    partita.txt          scritto dal SERVER:  PARTITA;id;seme;ATTESA|GIOCO
    ranking.txt          scritto dal SERVER:  la classifica (vedi protocollo.md)
    giocatori\NOME.txt   scritto da OGNI studente, una riga per tentativo:
                         id;NOME;1234;secondi
  Ogni studente scrive SOLO il suo file: niente conflitti in Git.
  Il server rilegge la cartella ogni 2 secondi e ricalcola tutto. }

{$mode objfpc}{$H+}

interface

uses
  SysUtils, Classes, mm_logica;

type
  { LATO SERVER }
  TMMFileServer = class
  private
    FTab: TMMTabella;
    FCartella: String;
    FLetti: TStringList;     // nomefile=righe gia' elaborate
  public
    constructor Create(ATab: TMMTabella; const ACartella: String);
    destructor Destroy; override;
    procedure ScriviFile;            // scrive partita.txt e ranking.txt
    function Poll: Integer;          // legge le righe nuove; ritorna quanti tentativi nuovi
    procedure Dimentica;             // dopo "Nuova partita"
    property Cartella: String read FCartella;
  end;

function MMScriviTesto(const NomeFile, Testo: String): Boolean;
function MMLeggiTesto(const NomeFile: String; out Testo: String): Boolean;
function MMAggiungiRiga(const NomeFile, Riga: String): Boolean;
function MMCartellaGiocatori(const Cartella: String): String;
function MMFileGiocatore(const Cartella, Nome: String): String;

implementation

function MMCartellaGiocatori(const Cartella: String): String;
begin
  Result := IncludeTrailingPathDelimiter(Cartella) + 'giocatori';
end;

function MMFileGiocatore(const Cartella, Nome: String): String;
begin
  Result := IncludeTrailingPathDelimiter(MMCartellaGiocatori(Cartella)) +
    PulisciNome(Nome) + '.txt';
end;

{ Lettura che non da' fastidio a chi sta scrivendo (fmShareDenyNone), con 3 tentativi }
function MMLeggiTesto(const NomeFile: String; out Testo: String): Boolean;
var
  F: TFileStream;
  k: Integer;
begin
  Testo := '';
  Result := False;
  for k := 1 to 3 do
  begin
    try
      if not FileExists(NomeFile) then Exit;
      F := TFileStream.Create(NomeFile, fmOpenRead or fmShareDenyNone);
      try
        SetLength(Testo, F.Size);
        if F.Size > 0 then F.ReadBuffer(Testo[1], F.Size);
      finally
        F.Free;
      end;
      Result := True;
      Exit;
    except
      Sleep(50);
    end;
  end;
end;

function MMScriviTesto(const NomeFile, Testo: String): Boolean;
var
  F: TFileStream;
begin
  Result := False;
```

```
  try
    ForceDirectories(ExtractFilePath(NomeFile));
    F := TFileStream.Create(NomeFile, fmCreate);
    try
      if Length(Testo) > 0 then F.WriteBuffer(Testo[1], Length(Testo));
    finally
      F.Free;
    end;
    Result := True;
  except
    Result := False;
  end;
end;

{ Aggiunge UNA riga in fondo (crea il file se non c'e'). Riprova 3 volte. }
function MMAggiungiRiga(const NomeFile, Riga: String): Boolean;
var
  F: TFileStream;
  k: Integer;
  s: String;
begin
  Result := False;
  s := Riga + LineEnding;
  for k := 1 to 3 do
  begin
    try
      ForceDirectories(ExtractFilePath(NomeFile));
      if FileExists(NomeFile) then
        F := TFileStream.Create(NomeFile, fmOpenWrite or fmShareDenyNone)
      else
        F := TFileStream.Create(NomeFile, fmCreate or fmShareDenyNone);
      try
        F.Seek(0, soEnd);
        F.WriteBuffer(s[1], Length(s));
      finally
        F.Free;
      end;
      Result := True;
      Exit;
    except
      Sleep(80);
    end;
  end;
end;

{ ------------------------------------------------------------ server }

constructor TMMFileServer.Create(ATab: TMMTabella; const ACartella: String);
begin
  inherited Create;
  FTab := ATab;
  FCartella := ACartella;
  FLetti := TStringList.Create;
  ForceDirectories(MMCartellaGiocatori(FCartella));
end;

destructor TMMFileServer.Destroy;
begin
  FLetti.Free;
  inherited Destroy;
end;

procedure TMMFileServer.Dimentica;
begin
  FLetti.Clear;
end;

procedure TMMFileServer.ScriviFile;
var
  base: String;
begin
  base := IncludeTrailingPathDelimiter(FCartella);
  FTab.Blocca;
  try
    MMScriviTesto(base + 'partita.txt', FTab.TestoPartita);
    MMScriviTesto(base + 'ranking.txt', FTab.TestoRanking);
  finally
    FTab.Sblocca;
  end;
end;

function TMMFileServer.Poll: Integer;
var
  sr: TSearchRec;
  cart, nomef, testo, riga, id: String;
  righe: TStringList;
  fatti, i, ultimo, idx, sec, esa, spo: Integer;
  esito: TEsito;
begin
  Result := 0;
```

```
  cart := IncludeTrailingPathDelimiter(MMCartellaGiocatori(FCartella));
  if FindFirst(cart + '*.txt', faAnyFile, sr) <> 0 then Exit;
  righe := TStringList.Create;
  try
    repeat
      nomef := cart + sr.Name;
      if not MMLeggiTesto(nomef, testo) then Continue;
      righe.Text := testo;
      ultimo := righe.Count;
      // l'ultima riga conta solo se il file finisce con "a capo" (altrimenti e' a meta')
      if (Length(testo) > 0) and not (testo[Length(testo)] in [#10, #13]) then
        ultimo := ultimo - 1;
      idx := FLetti.IndexOfName(sr.Name);
      if idx < 0 then fatti := 0 else fatti := StrToIntDef(FLetti.ValueFromIndex[idx], 0);
      i := fatti;
      while i < ultimo do
      begin
        riga := Trim(righe[i]);
        id := Campo(riga, 1);
        if (riga <> '') and (id = FTab.IdPartita) then
        begin
          sec := StrToIntDef(Campo(riga, 4), 0);
          FTab.Blocca;
          try
            esito := FTab.Tentativo(Campo(riga, 2), Campo(riga, 3), sec, esa, spo);
          finally
            FTab.Sblocca;
          end;
          if esito = esAttesa then Break;     // prima del Via: la riga resta in attesa
          if esito in [esOk, esVinto] then Result := Result + 1;
        end;
        i := i + 1;
      end;
      if idx < 0 then FLetti.Add(sr.Name + '=' + IntToStr(i))
      else FLetti.ValueFromIndex[idx] := IntToStr(i);
    until FindNext(sr) <> 0;
  finally
    righe.Free;
    FindClose(sr);
  end;
end;

end.
```

## mm_tcp.pas

Trasporto A: TCP con ssockets (thread)

```
unit mm_tcp;
{ MASTERMIND 2.0 - TRASPORTO A: RETE (TCP) - modo VELOCE e facoltativo.
  Usa SOLO ssockets, che e' dentro Free Pascal/Lazarus (niente pacchetti da
  installare, niente lNet, niente Indy).
  Ogni richiesta = una connessione: si manda UNA riga, si legge UNA riga, si chiude.
  Il server gira in un THREAD, cosi' la finestra non si blocca. }

{$mode objfpc}{$H+}

interface

uses
  SysUtils, Classes, sockets, ssockets, mm_logica;

type
  { SERVER: ascolta sulla porta e risponde con Tabella.Gestisci }
  TMMTcpServerThread = class(TThread)
  private
    FServer: TInetServer;
    FTabella: TMMTabella;
    FPorta: Word;
    procedure Connesso(Sender: TObject; Data: TSocketStream);
    procedure Inattivo(Sender: TObject);
  protected
    procedure Execute; override;
  public
    Errore: String;          // se non riesce ad aprire la porta
    Pronto: Boolean;         // True quando sta ascoltando
    constructor Create(ATabella: TMMTabella; APorta: Word);
  end;

{ CLIENT: manda una riga, restituisce la risposta. In caso di problemi: 'ERR;...' }
function MMTcpRichiesta(const Host: String; Porta: Word; const Linea: String;
  TimeoutMs: Integer = 3000): String;

type
  { Richiesta in un thread a parte: la finestra non si blocca mai.
    Si crea, e quando ha finito chiama AlFinire (nel thread principale). }
  TMMRichiestaAsync = class(TThread)
```

```
  private
    FHost: String;
    FPorta: Word;
    FLinea: String;
    FAlFinire: TNotifyEvent;
    procedure Fine;
  protected
    procedure Execute; override;
  public
    Risposta: String;
    constructor Create(const AHost: String; APorta: Word; const ALinea: String;
      AlFinire: TNotifyEvent);
  end;

implementation

{ ---- server ---- }

constructor TMMTcpServerThread.Create(ATabella: TMMTabella; APorta: Word);
begin
  FTabella := ATabella;
  FPorta := APorta;
  FreeOnTerminate := False;
  inherited Create(False);
end;

procedure TMMTcpServerThread.Connesso(Sender: TObject; Data: TSocketStream);
var
  c: Char;
  n: Integer;
  linea, risp: String;
begin
  try
    try
      Data.IOTimeout := 2000;           // un client muto non blocca il server
      linea := '';
      repeat
        n := Data.Read(c, 1);
        if (n = 1) and (c <> #10) and (c <> #13) then linea := linea + c;
      until (n <= 0) or (c = #10) or (Length(linea) > 200);
      if linea <> '' then
      begin
        FTabella.Blocca;
        try
          risp := FTabella.Gestisci(linea, NetAddrToStr(Data.RemoteAddress.sin_addr));
        finally
          FTabella.Sblocca;
        end;
        risp := risp + #10;
        Data.WriteBuffer(risp[1], Length(risp));
      end;
    except
      on E: Exception do ;              // un client che sbaglia non deve fermare il server
    end;
  finally
    Data.Free;
  end;
end;

procedure TMMTcpServerThread.Inattivo(Sender: TObject);
begin
  if Terminated then FServer.StopAccepting;
end;

procedure TMMTcpServerThread.Execute;
begin
  Errore := '';
  FServer := nil;
  try
    FServer := TInetServer.Create('0.0.0.0', FPorta);
    {$ifdef unix}FServer.ReuseAddress := True;{$endif}   // riavvio veloce senza "porta occupata"
    FServer.AcceptIdleTimeout := 250;    // ogni 1/4 di secondo controlla se deve fermarsi
    FServer.OnConnect := @Connesso;
    FServer.OnIdle := @Inattivo;
    Pronto := True;
    FServer.StartAccepting;             // resta qui finche' non arriva StopAccepting
  except
    on E: Exception do
    begin
      Errore := E.Message;
      Pronto := False;
    end;
  end;
  FServer.Free;
end;

{ ---- client ---- }

function MMTcpRichiesta(const Host: String; Porta: Word; const Linea: String;
  TimeoutMs: Integer): String;
```

```
var
  S: TInetSocket;
  c: Char;
  n: Integer;
  riga: String;
begin
  Result := '';
  try
    S := TInetSocket.Create(Host, Porta, TimeoutMs);
    try
      S.IOTimeout := TimeoutMs;
      riga := Linea + #10;
      S.WriteBuffer(riga[1], Length(riga));
      repeat
        n := S.Read(c, 1);
        if (n = 1) and (c <> #10) and (c <> #13) then Result := Result + c;
      until (n <= 0) or (c = #10) or (Length(Result) > 300);
    finally
      S.Free;
    end;
    if Result = '' then Result := 'ERR;nessuna risposta dal server';
  except
    on E: Exception do
      Result := 'ERR;server non raggiungibile (' + E.Message + ')';
  end;
end;

constructor TMMRichiestaAsync.Create(const AHost: String; APorta: Word;
  const ALinea: String; AlFinire: TNotifyEvent);
begin
  FHost := AHost;
  FPorta := APorta;
  FLinea := ALinea;
  FAlFinire := AlFinire;
  FreeOnTerminate := True;
  inherited Create(False);
end;

procedure TMMRichiestaAsync.Fine;
begin
  if Assigned(FAlFinire) then FAlFinire(Self);
end;

procedure TMMRichiestaAsync.Execute;
begin
  Risposta := MMTcpRichiesta(FHost, FPorta, FLinea);
  Synchronize(@Fine);
end;

end.
```

## client_unit.pas

Logica del client

```
unit client_unit;
{ MASTERMIND 2.0 - LOGICA DEL CLIENT (la parte dello studente).
  Nessuna finestra qui dentro: la form chiama queste funzioni.
  Due modi:  moFile = cartella condivisa / GitHub (PREDEFINITO, senza permessi di rete)
             moRete = TCP verso il PC del prof (veloce, se il firewall lo permette) }

{$mode objfpc}{$H+}

interface

uses
  SysUtils, Classes, mm_logica, mm_file, mm_tcp;

type
  TModoMM = (moFile, moRete);

  TMMClient = class
  private
    FIdVisto: String;      // partita di cui abbiamo gia' fissato lo "zero" del tempo
    FZero: Double;         // istante (orologio del PC) in cui abbiamo visto il Via
    function ProvaFile(const Tent: String): TRisposta;
  public
    Modo: TModoMM;
    Nome: String;
    Host: String;          // IP del PC del prof (modo rete)
    Porta: Word;           // 5050
    Cartella: String;      // cartella condivisa (modo file)
    Orologio: TFunzioneSecondi;
    Ultima: TSintesi;      // l'ultima sintesi conosciuta
    FallbackFile: Boolean; // True se la rete non andava e siamo passati ai file
    constructor Create;
    { modo rete: la riga da mandare e la lettura della risposta (la form puo' usare
```

```
      TMMRichiestaAsync per non bloccarsi) }
    function RigaProva(const Tent: String): String;
    function RigaStato: String;
    function ApplicaRisposta(const Linea: String): TRisposta;
    { fa tutto (in modo rete BLOCCA fino a 3 secondi: preferire l'Async) }
    function Prova(const Tent: String): TRisposta;
    { usata dalla form col thread: riceve la riga arrivata dalla rete; se la rete non
      andava e c'e' una cartella, rifa' il tentativo con i file (piano B automatico) }
    function RispostaDaRete(const Linea, Tent: String): TRisposta;
    function Aggiorna: TRisposta;      // aggiorna la console senza fare tentativi
    function Console: String;          // la frase da mostrare in una Label
    function TestoRisposta(const R: TRisposta): String;   // frase per l'esito del tentativo
    { Legge i due file di configurazione messi accanto al programma:
        server.txt    una riga:  IP:PORTA        (es. 192.168.10.5:5050)  -> modo rete
        cartella.txt  una riga:  percorso cartella condivisa                -> modo file
      Se ci sono tutti e due: parte in rete e, se il server non risponde, passa
      da solo ai file. Se c'e' solo cartella.txt: file. }
    function CaricaConfig(const CartellaProgramma: String): Boolean;
  end;

implementation

constructor TMMClient.Create;
begin
  inherited Create;
  Modo := moFile;
  Porta := 5050;
  Orologio := @SecondiOra;
  FillChar(Ultima, SizeOf(Ultima), 0);
end;

function TMMClient.RigaProva(const Tent: String): String;
begin
  Result := 'GUESS;' + PulisciNome(Nome) + ';' + Trim(Tent);
end;

function TMMClient.RigaStato: String;
begin
  Result := 'STATO;' + PulisciNome(Nome);
end;

function TMMClient.ApplicaRisposta(const Linea: String): TRisposta;
begin
  ParsaRisposta(Linea, Result);
  if (Result.Tipo = 'RESULT') or (Result.Tipo = 'WIN') or (Result.Tipo = 'STATO') then
    Ultima := Result.Sintesi;
end;

{ ---- modo file ---- }

{ ---- modo file ---- }

function TMMClient.ProvaFile(const Tent: String): TRisposta;
var
  base, testoP, testoR, mio, riga, segreto, id, tutto: String;
  seme, via: Int64;
  fase: TFase;
  sint: TSintesi;
  righe: TStringList;
  i, mie, ultimoSec, esa, spo, sec: Integer;
  risolto: Boolean;
begin
  Result := Default(TRisposta);
  base := IncludeTrailingPathDelimiter(Cartella);
  if not CodiceValido(Trim(Tent)) then
  begin
    Result.Tipo := 'ERR';
    Result.Messaggio := 'codice non valido (servono 4 cifre da 1 a 6)';
    Exit;
  end;
  if (PulisciNome(Nome) = '') then
  begin
    Result.Tipo := 'ERR';
    Result.Messaggio := 'scrivi il tuo nome';
    Exit;
  end;
  if not MMLeggiTesto(base + 'partita.txt', testoP) then
  begin
    Result.Tipo := 'ERR';
    Result.Messaggio := 'non trovo partita.txt nella cartella (sincronizza o controlla il percorso)';
    Exit;
  end;
  // partita.txt ha la stessa forma di una riga PARTITA di ranking.txt
  sint := SintesiDaRanking(testoP, Nome, fase, id, seme);
  if (fase <> faGioco) or (seme = 0) then
  begin
    Result.Tipo := 'WAIT';
    Exit;
  end;
  segreto := CodiceDaSeme(seme);
```

```
  mio := MMFileGiocatore(Cartella, Nome);
  // rileggo il MIO file: quanti tentativi ho fatto in questa partita? ho gia' vinto?
  mie := 0;
  ultimoSec := 0;
  risolto := False;
  righe := TStringList.Create;
  try
    if MMLeggiTesto(mio, tutto) then righe.Text := tutto;
    for i := 0 to righe.Count - 1 do
    begin
      riga := Trim(righe[i]);
      if Campo(riga, 1) = id then
      begin
        mie := mie + 1;
        ultimoSec := StrToIntDef(Campo(riga, 4), 0);
        if Campo(riga, 3) = segreto then risolto := True;
      end;
    end;
  finally
    righe.Free;
  end;
  // lo "zero" del tempo: il Via scritto dal server in partita.txt
  if FIdVisto <> id then
  begin
    FIdVisto := id;
    via := StrToInt64Def(Campo(Trim(testoP), 5), 0);      // ora del Via secondo il server
    if (via > 0) and (Orologio() - via >= 0) and (Orologio() - via < 14400) then
      FZero := via                                        // orologi d'accordo: conta dal Via vero
    else
      FZero := Orologio() - ultimoSec;                    // orologi sballati: conta da quando lo vedo io
  end;
  sec := Round(Orologio() - FZero);
  Valuta(segreto, Trim(Tent), esa, spo);
  if not risolto then
  begin
    MMAggiungiRiga(mio, id + ';' + PulisciNome(Nome) + ';' + Trim(Tent) + ';' + IntToStr(sec));
    mie := mie + 1;
    risolto := (esa = 4);
  end
  else
  begin
    esa := 4;      // ha gia' vinto: i tentativi in piu' non contano
    spo := 0;
  end;
  // la classifica la vedo da ranking.txt (puo' essere in ritardo)
  sint := Default(TSintesi);
  if MMLeggiTesto(base + 'ranking.txt', testoR) then
    sint := SintesiDaRanking(testoR, Nome, fase, id, seme);
  sint.Tentativi := mie;
  if not risolto then sint.Posizione := 0;
  Result.Esatte := esa;
  Result.Spostate := spo;
  Result.Sintesi := sint;
  if risolto then
  begin
    Result.Tipo := 'WIN';
    Result.Secondi := sec;
  end
  else
    Result.Tipo := 'RESULT';
  Ultima := sint;
end;

function TMMClient.RispostaDaRete(const Linea, Tent: String): TRisposta;
begin
  Result := ApplicaRisposta(Linea);
  if (Result.Tipo = 'ERR') and (Pos('server non raggiungibile', Result.Messaggio) > 0)
     and (Cartella <> '') then
  begin
    Modo := moFile;                // piano B automatico
    FallbackFile := True;
    Result := ProvaFile(Tent);
  end;
end;

function TMMClient.Prova(const Tent: String): TRisposta;
begin
  if Modo = moFile then
    Result := ProvaFile(Tent)
  else
    Result := RispostaDaRete(MMTcpRichiesta(Host, Porta, RigaProva(Tent)), Tent);
end;

function LeggiPrimaRiga(const Nomefile: String): String;
var
  t: String;
  L: TStringList;
begin
  Result := '';
  if not MMLeggiTesto(Nomefile, t) then Exit;
```

```
  L := TStringList.Create;
  try
    L.Text := t;
    if L.Count > 0 then Result := Trim(L[0]);
  finally
    L.Free;
  end;
end;

function TMMClient.CaricaConfig(const CartellaProgramma: String): Boolean;
var
  base, srv, cart: String;
  k: Integer;
begin
  base := IncludeTrailingPathDelimiter(CartellaProgramma);
  srv := LeggiPrimaRiga(base + 'server.txt');
  cart := LeggiPrimaRiga(base + 'cartella.txt');
  Result := False;
  Cartella := cart;
  if srv <> '' then
  begin
    k := Pos(':', srv);
    if k = 0 then
    begin
      Host := srv;
      Porta := 5050;
    end
    else
    begin
      Host := Trim(Copy(srv, 1, k - 1));
      Porta := StrToIntDef(Trim(Copy(srv, k + 1, 10)), 5050);
    end;
    Modo := moRete;
    Result := True;
  end
  else if cart <> '' then
  begin
    Modo := moFile;
    Result := True;
  end;
end;

function TMMClient.Aggiorna: TRisposta;
var
  testo, id: String;
  fase: TFase;
  seme: Int64;
  s: TSintesi;
  mie: Integer;
begin
  Result := Default(TRisposta);
  if Modo = moRete then
  begin
    Result := ApplicaRisposta(MMTcpRichiesta(Host, Porta, RigaStato));
    Exit;
  end;
  if MMLeggiTesto(IncludeTrailingPathDelimiter(Cartella) + 'ranking.txt', testo) then
  begin
    mie := Ultima.Tentativi;
    s := SintesiDaRanking(testo, Nome, fase, id, seme);
    if s.Tentativi < mie then s.Tentativi := mie;     // i miei tentativi li conosco meglio io
    Ultima := s;
    Result.Tipo := 'STATO';
    Result.Sintesi := s;
    if fase = faAttesa then Result.Messaggio := 'ATTESA' else Result.Messaggio := 'GIOCO';
  end
  else
  begin
    Result.Tipo := 'ERR';
    Result.Messaggio := 'non trovo ranking.txt nella cartella';
  end;
end;

function TMMClient.Console: String;
begin
  Result := TestoConsole(Ultima);
end;

function TMMClient.TestoRisposta(const R: TRisposta): String;
begin
  if R.Tipo = 'RESULT' then
    Result := 'Esatte: ' + IntToStr(R.Esatte) + '  Spostate: ' + IntToStr(R.Spostate)
  else if R.Tipo = 'WIN' then
    Result := 'BRAVO! Indovinato in ' + IntToStr(R.Sintesi.Tentativi) + ' tentativi'
  else if R.Tipo = 'WAIT' then
    Result := 'Aspetta il Via del prof...'
  else
    Result := 'Problema: ' + R.Messaggio;
end;
```

```
end.
```

## unit1_client.pas

Il Mastermind dello studente con la scheda Classifica (sostituisce unit1.pas)

```
unit Unit1;
{ Il Mastermind NORMALE della 3INF (ora2_unit1.pas) con in piu' la scheda "Classifica".
  Il gioco ora e' contro il codice del SERVER del prof, uguale per tutta la classe.
  Cosa e' cambiato rispetto a prima: vedi i commenti "// NUOVO" e "// CAMBIATO". }

{$mode objfpc}{$H+}

interface

uses
  Classes, SysUtils, Forms, Controls, Graphics, Dialogs, StdCtrls, ComCtrls, ExtCtrls,
  mm_logica, mm_tcp, client_unit;                // NUOVO: i 4 file del Mastermind 2.0

type
  TForm1 = class(TForm)
    PageControl1: TPageControl;   // le schede
    TabSheet1: TTabSheet;         // scheda Gioco
    TabSheet2: TTabSheet;         // scheda Regole
    TabSheet3: TTabSheet;         // scheda Classifica (prima si chiamava Record)
    Button1: TButton;             // Prova
    Button2: TButton;             // Nuova partita (ora la decide il prof: non fa niente)
    Edit1: TEdit;                 // il tuo tentativo
    Edit2: TEdit;                 // NUOVO: il tuo nome (nella scheda Gioco)
    Label1: TLabel;               // risultato del tentativo
    Label2: TLabel;               // (il codice ora e' segreto: resta li' ma non lo mostra)
    Label3: TLabel;               // le regole (scheda Regole)
    Label4: TLabel;               // CAMBIATO: la console della classifica (scheda Classifica)
    Memo1: TMemo;                 // i tentativi
    procedure Button1Click(Sender: TObject);
    procedure Button2Click(Sender: TObject);
    procedure FormCreate(Sender: TObject);
  private
    Cl: TMMClient;                // NUOVO: tutta la parte di rete/file sta qui dentro
    Orologio3s: TTimer;           // NUOVO: ogni 3 secondi aggiorna la classifica (creato nel codice)
    UltimoTentativo: String;
    procedure Timer1Timer(Sender: TObject);
    procedure MostraEsito(const R: TRisposta; const Tent: String);
    procedure RispostaTentativo(Sender: TObject);
    procedure RispostaStato(Sender: TObject);
  public
  end;

var
  Form1: TForm1;

implementation

{$R *.lfm}

procedure TForm1.FormCreate(Sender: TObject);
begin
  Label3.Caption := 'Il server del prof pensa 4 cifre da 1 a 6, uguali per tutti. ' +
    'Tu provi. "Esatte" = cifra giusta al posto giusto. ' +
    '"Spostate" = cifra giusta ma al posto sbagliato. Vince chi usa meno tentativi.';
  Label4.Caption := 'Scrivi il tuo nome e aspetta il Via del prof';
  Cl := TMMClient.Create;
  // legge server.txt (IP:PORTA) e/o cartella.txt accanto al programma
  if not Cl.CaricaConfig(ExtractFilePath(Application.ExeName)) then
    Label1.Caption := 'Manca il file server.txt o cartella.txt accanto al programma';
  Label2.Caption := 'Codice segreto: ????';
  Orologio3s := TTimer.Create(Self);        // il timer nasce qui, non serve metterlo sulla Form
  Orologio3s.Interval := 3000;
  Orologio3s.OnTimer := @Timer1Timer;
  Orologio3s.Enabled := True;
end;

procedure TForm1.Button2Click(Sender: TObject);   // Nuova partita
begin
  Label1.Caption := 'La nuova partita la fa partire il prof';
end;

{ Scrive nel Label1 e nel Memo cosa e' successo }
procedure TForm1.MostraEsito(const R: TRisposta; const Tent: String);
begin
  Label1.Caption := Cl.TestoRisposta(R);
  if (R.Tipo = 'RESULT') or (R.Tipo = 'WIN') then
    Memo1.Lines.Add(IntToStr(R.Sintesi.Tentativi) + ') ' + Tent + ' -> esatte: ' +
      IntToStr(R.Esatte) + '  spostate: ' + IntToStr(R.Spostate));
  Label4.Caption := Cl.Console;
end;
```

```
procedure TForm1.Button1Click(Sender: TObject);   // Prova
begin
  Cl.Nome := Edit2.Text;
  UltimoTentativo := Edit1.Text;
  if Length(UltimoTentativo) <> 4 then
  begin
    Label1.Caption := 'Scrivi 4 cifre!';
    Exit;
  end;
  if Cl.Modo = moRete then
  begin
    Label1.Caption := 'Invio al server...';
    // la richiesta parte in un THREAD: la finestra non si blocca
    TMMRichiestaAsync.Create(Cl.Host, Cl.Porta, Cl.RigaProva(UltimoTentativo), @RispostaTentativo);
  end
  else
    MostraEsito(Cl.Prova(UltimoTentativo), UltimoTentativo);   // i file sono veloci
  Edit1.Clear;
end;

procedure TForm1.RispostaTentativo(Sender: TObject);   // chiamata quando il server ha risposto
var
  testo: String;
begin
  testo := TMMRichiestaAsync(Sender).Risposta;
  MostraEsito(Cl.RispostaDaRete(testo, UltimoTentativo), UltimoTentativo);
end;

procedure TForm1.RispostaStato(Sender: TObject);
begin
  Cl.ApplicaRisposta(TMMRichiestaAsync(Sender).Risposta);
  Label4.Caption := Cl.Console;
end;

procedure TForm1.Timer1Timer(Sender: TObject);   // ogni 3 secondi
begin
  Cl.Nome := Edit2.Text;
  if Cl.Nome = '' then Exit;
  if Cl.Modo = moRete then
    TMMRichiestaAsync.Create(Cl.Host, Cl.Porta, Cl.RigaStato, @RispostaStato)
  else
  begin
    Cl.Aggiorna;
    Label4.Caption := Cl.Console;
  end;
end;

end.
```

## server_mastermind.pas

Il server a console del prof

```
program server_mastermind;
{ MASTERMIND 2.0 - SERVER (programma a console, gira sul PC del prof).
  Tasti:  V = Via!   N = Nuova partita (nuovo codice)   R = ridisegna
          I = rileggi ip-nomi.txt   Q = esci
  Opzioni (tutte facoltative), per esempio:  server_mastermind --porta=5050 --cartella=partita
    --porta=N        porta TCP (predefinita 5050)
    --cartella=PATH  cartella dei file (predefinita: sottocartella "partita")
    --senza-rete     solo file, niente TCP (nessun permesso del firewall)
    --seme=N         codice fisso (per le prove)
    --via            parte subito senza aspettare il tasto V
    --senza-tasti    non legge la tastiera (per le prove automatiche)
    --durata=SEC     si ferma da solo dopo SEC secondi (per le prove)
  Comandi anche da file: scrivere VIA oppure NUOVA dentro  <cartella>\comando.txt }

{$mode objfpc}{$H+}

uses
  {$ifdef unix}cthreads,{$endif}
  SysUtils, Classes, Crt, mm_logica, mm_file, mm_tcp;

var
  Tab: TMMTabella;
  FS: TMMFileServer;
  Srv: TMMTcpServerThread;
  porta: Word;
  cartella, fileIp, ultimoQuadro, quadro, cmd, t: String;
  senzaRete, senzaTasti, avviaVia: Boolean;
  durata: Integer;
  i, nuovi: Integer;
  seme: Int64;
  inizio, ultimoPoll: Double;
  fine: Boolean;
  ultimoRanking: String;
```

```
  exeDir: String;

function Opzione(const Nome: String; out Valore: String): Boolean;
var
  k: Integer;
begin
  Result := False;
  for k := 1 to ParamCount do
    if (ParamStr(k) = Nome) or (Copy(ParamStr(k), 1, Length(Nome) + 1) = Nome + '=') then
    begin
      Valore := Copy(ParamStr(k), Length(Nome) + 2, 500);
      Result := True;
      Exit;
    end;
end;

procedure CaricaIp;
var
  txt: String;
begin
  if MMLeggiTesto(fileIp, txt) then Tab.CaricaIpNomi(txt) else Tab.IpNomi.Clear;
end;

procedure NuovaPartita;
begin
  Tab.Blocca;
  try
    Tab.NuovaPartita(seme);     // seme = -1 -> casuale
    seme := -1;
  finally
    Tab.Sblocca;
  end;
  FS.Dimentica;
  FS.ScriviFile;
end;

procedure FaiVia;
begin
  Tab.Blocca;
  try
    Tab.Via;
  finally
    Tab.Sblocca;
  end;
  FS.ScriviFile;
end;

function Quadro_: String;
begin
  Tab.Blocca;
  try
    Result := 'MASTERMIND 2.0 - SERVER   [V] Via!  [N] Nuova partita  [R] ridisegna  [I] ip-nomi  [Q] esci' + LineEndi
ng;
    if senzaRete then Result := Result + 'Rete: SPENTA (solo file)  '
    else if Srv.Pronto then Result := Result + 'Rete: in ascolto sulla porta ' + IntToStr(porta) + '  '
    else Result := Result + 'Rete: NON attiva ' + Srv.Errore + '  ';
    Result := Result + 'Cartella: ' + cartella + LineEnding + LineEnding +
      Tab.TestoClassifica + LineEnding + Tab.TestoPresenze;
  finally
    Tab.Sblocca;
  end;
end;

procedure Disegna(Forza: Boolean);
begin
  quadro := Quadro_;
  if Forza or (quadro <> ultimoQuadro) then
  begin
    ultimoQuadro := quadro;
    if not senzaTasti then ClrScr else WriteLn('------------------------------------------------');
    Write(quadro);
  end;
end;

begin
  Tab := TMMTabella.Create;
  exeDir := ExtractFilePath(ParamStr(0));
  porta := 5050;
  if Opzione('--porta', t) then porta := StrToIntDef(t, 5050);
  if not Opzione('--cartella', cartella) then cartella := exeDir + 'partita';
  fileIp := exeDir + 'ip-nomi.txt';
  if Opzione('--ipnomi', t) then fileIp := t;
  senzaRete := Opzione('--senza-rete', t);
  senzaTasti := Opzione('--senza-tasti', t);
  avviaVia := Opzione('--via', t);
  durata := 0;
  if Opzione('--durata', t) then durata := StrToIntDef(t, 0);
  seme := -1;
  if Opzione('--seme', t) then seme := StrToInt64Def(t, -1);
```

```
  FS := TMMFileServer.Create(Tab, cartella);
  CaricaIp;
  NuovaPartita;
  Srv := nil;
  if not senzaRete then
  begin
    Srv := TMMTcpServerThread.Create(Tab, porta);
    Sleep(300);
  end;
  if avviaVia then FaiVia;
  inizio := SecondiOra;
  ultimoPoll := 0;
  fine := False;
  ultimoQuadro := '';
  ultimoRanking := '';
  repeat
    // 1. tastiera
    if not senzaTasti then
      while KeyPressed do
        case UpCase(ReadKey) of
          'V': FaiVia;
          'N': begin NuovaPartita; ultimoQuadro := ''; end;
          'R': ultimoQuadro := '';
          'I': CaricaIp;
          'Q': fine := True;
        end;
    // 2. comando da file
    if MMLeggiTesto(IncludeTrailingPathDelimiter(cartella) + 'comando.txt', cmd) then
    begin
      DeleteFile(IncludeTrailingPathDelimiter(cartella) + 'comando.txt');
      cmd := UpperCase(Trim(cmd));
      if cmd = 'VIA' then FaiVia
      else if cmd = 'NUOVA' then NuovaPartita
      else if cmd = 'FINE' then fine := True;
    end;
    // 3. ogni 2 secondi rilegge i file dei ragazzi
    if SecondiOra - ultimoPoll >= 2 then
    begin
      ultimoPoll := SecondiOra;
      nuovi := FS.Poll;
      if nuovi > 0 then FS.ScriviFile;
    end;
    // 4. se la rete ha cambiato la classifica, riscrivo i file (cosi' chi usa i file la vede)
    Tab.Blocca;
    try
      t := Tab.TestoRanking;
    finally
      Tab.Sblocca;
    end;
    if t <> ultimoRanking then
    begin
      ultimoRanking := t;
      FS.ScriviFile;
    end;
    Disegna(False);
    Sleep(100);
    if (durata > 0) and (SecondiOra - inizio > durata) then fine := True;
  until fine;
  FS.ScriviFile;
  if Srv <> nil then
  begin
    Srv.Terminate;
    Srv.WaitFor;
    Srv.Free;
  end;
  FS.Free;
  Tab.Free;
end.
```